

Secure Hash Algorithm (SHA)

SHA (Secure Hash Algorithm) ist eine Familie kryptografischer Hash-Funktionen, die von der National Security Agency (NSA) entwickelt und vom National Institute of Standards and Technology (NIST) veröffentlicht wurden. SHA wird hauptsächlich verwendet, um Daten zu einer festen Länge zu hashen, was bedeutet, dass eine beliebige Eingabegröße (Nachricht) auf eine feste Ausgabegröße (z.B. 256 oder 512 Bits) abgebildet wird. In Java kann SHA für verschiedene Anwendungen verwendet werden, darunter Datenintegrität, digitale Signaturen und Passwortsicherung.

Aspekt	Beschreibung
SHA-Familie	Die SHA-Familie besteht aus mehreren Versionen:
	SHA-1: Gibt eine 160-Bit-Hash-Ausgabe zurück (früher weit verbreitet, aber mittlerweile als unsicher gelten, da Kollisionen gefunden wurden). SHA-2: Besteht aus mehreren Varianten (SHA-224, SHA-256, SHA-384, SHA-512). Die am meisten verwendeten Varianten sind SHA-256 und SHA-512. SHA-3: Eine neuere Variante, die auf einem anderen Algorithmus basiert (Keccak).
Verwendungszwecke von SHA	Integrität: Überprüfung, ob Daten verändert wurden. Passwort-Hashing: Schutz von Passwörtern. Digitale Signaturen: Bei der Erstellung von Signaturen wird häufig eine Hash-Funktion verwendet, um die Nachricht zu komprimieren. Prüfwerte und Fingerabdrücke: Überprüfen von Datenintegrität.
Eigenschaften von SHA	Deterministisch: Für dieselbe Eingabe wird immer derselbe Hash-Wert erzeugt. Schnell: SHA-Algorithmen sind schnell und effizient. Kollisionsresistenz: Es ist schwierig, zwei verschiedene Eingaben zu finden, die denselben Hash-Wert erzeugen. Avalanche-Effekt: Eine kleine Änderung der Eingabe führt zu einer dramatischen Veränderung des Hash-Werts.
Hash-Wert	Der Hash-Wert (oder Digest) ist die Ausgabe der Hash-Funktion und hat bei SHA-256 eine Länge von 256 Bits, bei SHA-512 eine Länge von 512 Bits. Dies ist eine eindeutige Repräsentation der Eingabe.
Kollisionsresistenz	Die Fähigkeit eines Hash-Algorithmus, Kollisionen zu vermeiden, bedeutet, dass es praktisch unmöglich ist, zwei verschiedene Datenmengen zu finden, die denselben Hash-Wert erzeugen. SHA-1 hat diese Eigenschaft jedoch verloren, während SHA-2 und SHA-3 noch als sicher gelten.
Verwendung in Java	In Java wird SHA durch die Java Cryptography Extension (JCE) unterstützt, insbesondere über die Klassen <code>MessageDigest</code> und <code>DigestInputStream</code> .

Methode/Klasse	Beschreibung
<code>MessageDigest</code>	<code>MessageDigest</code> ist die zentrale Java-Klasse, die die Hash-Funktionen wie SHA-1, SHA-256 und SHA-512 bereitstellt.
<code>MessageDigest.getInstance(String algorithm)</code>	Diese Methode wird verwendet, um eine Instanz von <code>MessageDigest</code> zu erstellen, die den angegebenen Algorithmus unterstützt, z.B. "SHA-256" oder "SHA-512".
<code>MessageDigest.update(byte[] input)</code>	Diese Methode fügt Eingabedaten zu einem bestehenden Hash hinzu. Sie kann mehrmals aufgerufen werden, um den Hash auf einem Stück zu berechnen, wenn die Eingabedaten groß sind.
<code>MessageDigest.digest()</code>	Diese Methode gibt den berechneten Hash-Wert (Digest) als Byte-Array zurück.
<code>DigestInputStream</code>	Diese Klasse ist ein Wrapper um einen InputStream, der die Daten in einen Hash berechnet, während sie vom Stream gelesen werden. Sie wird verwendet, wenn Daten direkt aus einer Datei oder einem Netzwerk-Stream gehasht werden sollen.
<code>MessageDigest.reset()</code>	Setzt den aktuellen Zustand des <code>MessageDigest</code> zurück, sodass er für neue Berechnungen wiederverwendet werden kann.
<code>Base64</code>	Die Klasse <code>Base64</code> kann verwendet werden, um den Hash-Wert in ein für den Menschen lesbares Format zu kodieren, beispielsweise bei der Anzeige von Hash-Werten oder deren Speicherung.

Methode	Beschreibung
<code>MessageDigest.getInstance("SHA-256")</code>	Diese Methode gibt eine Instanz von <code>MessageDigest</code> zurück, die den SHA-256-Algorithmus verwendet. Sie wird verwendet, um Hash-Werte mit SHA-256 zu berechnen.
<code>MessageDigest.update(byte[] input)</code>	Diese Methode fügt Daten hinzu, die in den Hash-Prozess einfließen. Die Eingabedaten werden als Byte-Array übergeben. Wenn die zu verarbeitenden Daten aus mehreren Teilen bestehen, kann diese Methode mehrfach aufgerufen werden.

Methode	Beschreibung
<code>MessageDigest.digest()</code>	Diese Methode gibt den endgültigen Hash-Wert der eingegebenen Daten zurück. Der Wert ist in Byte-Array-Form, und der Hash-Wert hängt vom verwendeten SHA-Algorithmus ab (z.B. 256 Bit bei SHA-256).
<code>Base64.getEncoder().encodeToString(byte[] src)</code>	Diese Methode kann verwendet werden, um den berechneten Hash-Wert in Base64 zu kodieren, sodass er in einer Textform wie z.B. in einer Datei oder für die Ausgabe auf einer Webseite gespeichert oder angezeigt werden kann.
<code>MessageDigest.reset()</code>	Diese Methode setzt die <code>MessageDigest</code> -Instanz auf den ursprünglichen Zustand zurück, sodass der Hash-Prozess für neue Eingabedaten neu gestartet werden kann.

SHA-Algorithmus	Ausgabegröße	Verwendung
SHA-1	160 Bits	Früher weit verbreitet, jedoch durch Kollisionen unsicher. Wird oft durch SHA-256 ersetzt.
SHA-256	256 Bits	Sehr sicher und weit verbreitet, insbesondere in modernen Anwendungen und Blockchain-Technologien (z.B. Bitcoin).
SHA-512	512 Bits	Bietet eine noch höhere Sicherheit als SHA-256 und wird in Fällen verwendet, in denen ein größerer Hash-Wert erforderlich ist (z.B. in bestimmten Sicherheitsprotokollen).
SHA-3	224, 256, 384, 512 Bits	Die neueste Version der SHA-Familie, die auf einem völlig anderen Algorithmus (Keccak) basiert. SHA-3 bietet eine höhere Sicherheit und wird in modernen Anwendungen bevorzugt, wenn absolute Sicherheit erforderlich ist.

Schritt	Beschreibung
1. <code>MessageDigest</code> Instanz erstellen	<code>MessageDigest md = MessageDigest.getInstance("SHA-256");</code> Diese Methode wird verwendet, um eine Instanz von <code>MessageDigest</code> zu erstellen, die den gewünschten SHA-Algorithmus verwendet.
2. Daten hinzufügen	<code>md.update(inputData.getBytes());</code> Fügt die Eingabedaten zum Hash-Prozess hinzu. Diese Methode kann mehrfach aufgerufen werden, um größere Datenmengen zu verarbeiten.

Schritt	Beschreibung
3. Hash-Wert berechnen	<code>byte[] hash = md.digest();</code> Berechnet den endgültigen Hash-Wert. Die Ausgabe ist ein Byte-Array.
4. Hash-Wert anzeigen	<code>String encoded = Base64.getEncoder().encodeToString(hash);</code> Kodiert den Hash-Wert in Base64, um ihn als Text darzustellen.

Zusammenfassung

SHA (Secure Hash Algorithm) ist eine Familie von Hash-Algorithmen, die in verschiedenen Sicherheitsanwendungen verwendet wird, von der Datenintegrität bis zur digitalen Signatur.

SHA-256 und **SHA-512** sind die am häufigsten verwendeten Varianten von SHA, mit SHA-3 als neuerer, sicherer Option.

In Java wird **SHA** über die `MessageDigest`-Klasse verwendet, die die Berechnung von Hash-Werten ermöglicht.

Kollisionsresistenz und **Sicherheit** sind zentrale Eigenschaften von SHA, wobei SHA-1 mittlerweile als unsicher gilt.

Der Hash-Wert kann durch **Base64** kodiert werden, um ihn menschenlesbar zu machen.
